TASK 6: Design the Inheritance Hierarchy

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Superclass and subclasses

Vehicle is the superclass. TwoWheeler and Car are direct subclasses. ElectricCar extends Car, so it is a subclass of Car and also of Vehicle. The hierarchy uses hierarchical inheritance (TwoWheeler and Car both come from Vehicle) and multilevel inheritance (Vehicle to Car to ElectricCar). Users (student, staff and visitor) can follow the same hierarchical pattern from a Person superclass, which is not needed for the code but fits the scenario.

What is inherited and what is added

Class	Inherited from above	Added in this class
Vehicle	Object methods	regNo, getRegNo(), calculateFee(), equals(), hashCode()
TwoWheeler	regNo, calculateFee()	ratePerHour() = 10
Car	regNo, calculateFee()	ratePerHour() = 30
ElectricCar	Everything in Car and Vehicle	chargingFee(kWh) = 12 per unit (from Chargeable)

How super is used

super(regNo) in each constructor passes the registration number up to the superclass, so Vehicle sets the field once. super is also used to call a superclass method when a subclass needs the original behaviour as part of its own result.

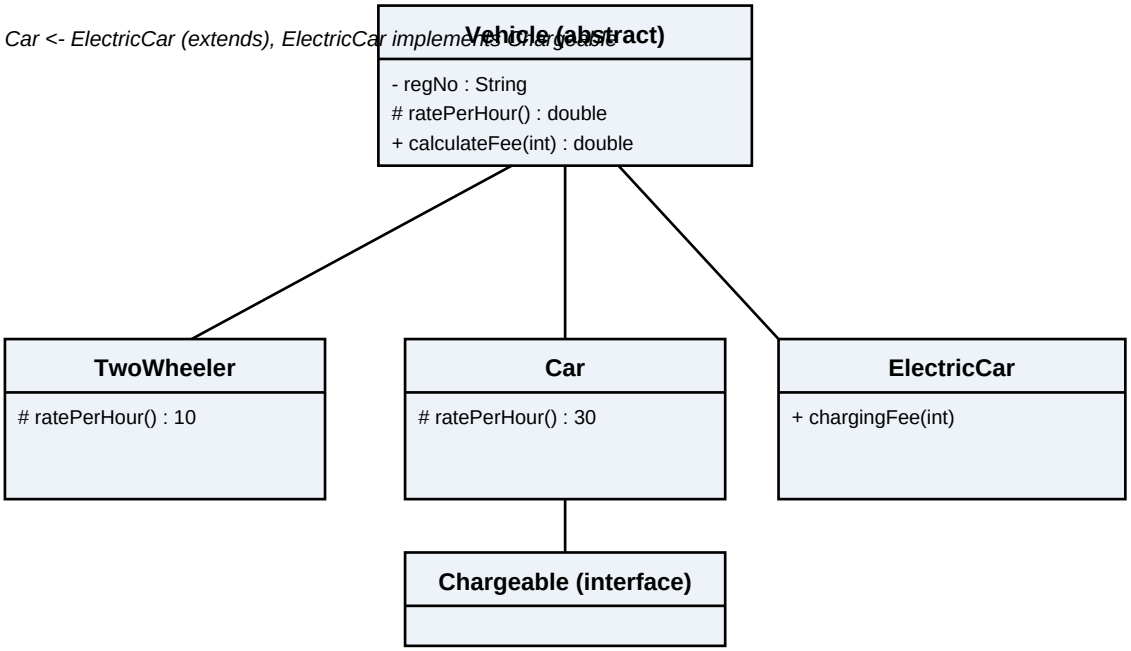
```
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }   // pass the number up
}
```

Constructor order: trace for new ElectricCar("TN03C3")

- 1. The ElectricCar constructor starts and calls super(regNo) first.
- 2. The Car constructor starts and calls super(regNo).
- 3. The Vehicle constructor starts. It calls the implicit Object() constructor first.
- 4. Object() finishes, then the Vehicle body runs and sets regNo.
- 5. The Car body runs (nothing extra here).
- 6. The ElectricCar body runs last. The object is now fully built.

The order is always top to bottom in the hierarchy (superclass first), so every superclass field is ready before a subclass uses it.

UML class diagram



Alternative: composition instead of inheritance

With composition, a single Vehicle class has a VehicleType object that holds the rate, for example new Vehicle(regNo, CAR_TYPE). Adding a new type then needs only a new VehicleType object (no new class), and the fee is found by asking the type object for its rate. The fee lookup is still O(1). Space per vehicle is one extra reference to the type object, which can be shared by many vehicles. Inheritance gives each type its own behaviour (for example, the van's peak rule) by overriding a method, while composition needs a type object with a rule field or a strategy object for that behaviour. For this assignment inheritance is the better fit, because the e-bike and van rules are different in shape, not just in rate.